

# Multimodal Conversational System

**Instructor:** Varun Khatri

{ik} INTERVIEW KICKSTART

# Introduction

**Varun | Software Engineer, Agentic AI @ Microsoft**

*20+ Years of Evolving With the Industry*




**20+ Years**
Software Engineer, since 2005

**4+ Years**
At Microsoft

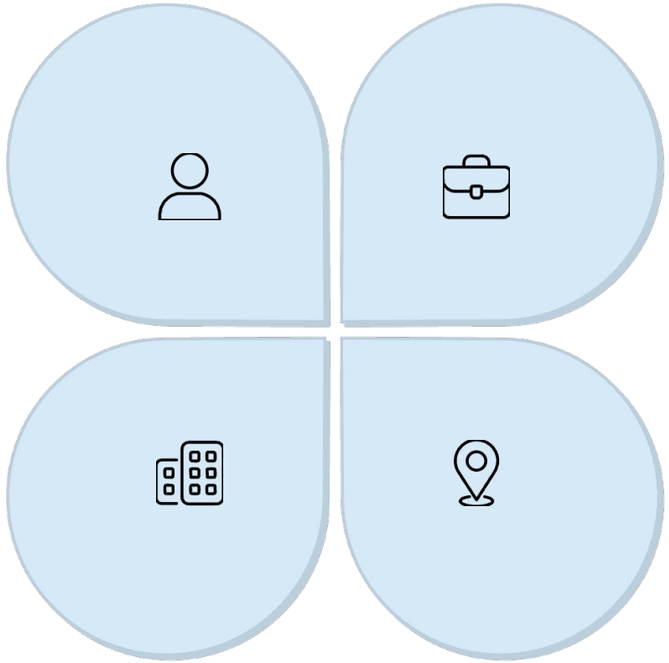
{ik} INTERVIEW KICKSTART

# Welcome to today's class, before we begin,

## pop into the chat:

**Your Name**

**Your Role**



**Your Company**

**Your Location**

# Optimize Your Experience

### Interact with Instructors

Join Sunday live class to ask questions and get clarifications.

### Engage with Assessments

Complete assignments and MCQs. Get feedback during Thursday review sessions.

### Utilize Your Resources

Make the most of all available resources to maximize your learning experience.

{ik} INTERVIEW KICKSTART



# Are you using all these IK support features?



**Uplevel Resources**

Contains all the learning resources - videos, MCQs, assignments.



**Post-Class Videos**

Optional videos for additional reference.



**Technical Coaching**

Sessions on Wednesday to clarify all your queries.



**Discord Community**

For peer learning and collaboration.



**TA Assistance**

TAs help answer queries in the live class.



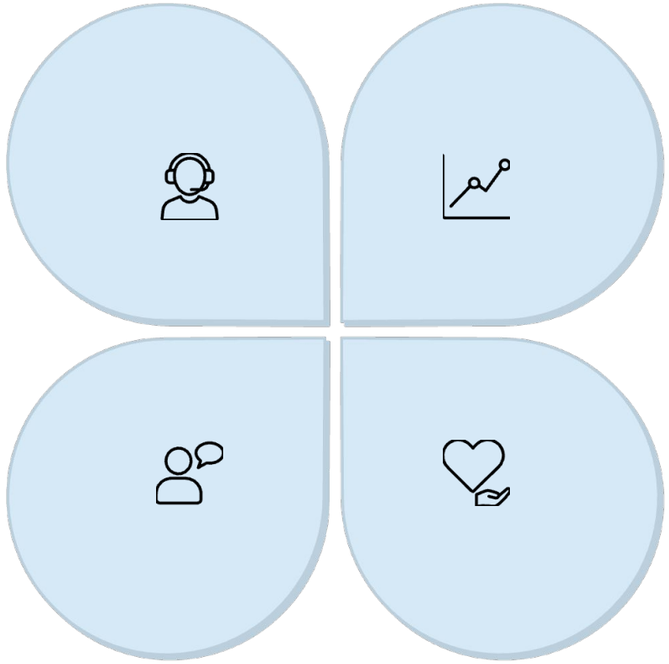
**Support Tickets**

Raise them any time you want any clarifications/tech concerns, etc on UpLevel.

# Learning Objectives

Voice Agents

Concept of Sub-Graphs in LangGraph



Multi-Modal Conversational Agents

Human In The Loop (HITL)

{ik} INTERVIEW KICKSTART

# Before We Begin

We assume that from previous lectures you already have understanding of:

- **LLMs**
- **RAG in Agentic Workflows**
- **Tools & Tool Calls**
- **LangChain**
- **LangGraph**
- **Parallelisation**
- **Multi-Agent Systems**
- **Vector Databases**
- **Persistence**

{ik} INTERVIEW KICKSTART

# Voice Agents

AI agents that engage in natural spoken conversations — combining speech recognition, NLP, generative AI, and text-to-speech.



**Customer Support**

Handle queries, resolve issues, escalate when needed



**Personal Assistants**

Manage tasks, reminders, and information on demand



**Hands-Free Interfaces**

Ideal for driving, accessibility, and industrial settings



**Healthcare Triage**

Collect symptoms, guide patients to the right care



**Sales & Outreach**

Qualify leads and book meetings via phone



**Language Learning**

Conversational practice with real-time corrections



**Banking & Finance**

Account queries, fraud alerts, transaction support



**Smart Home Control**

Voice-driven device and environment management

*...and many more real-world applications across every industry.*

{ik} INTERVIEW KICKSTART

# How Do Voice Agents Work?

Every voice agent handles three core tasks — the difference lies in how they're sequenced and coupled.

01

**Listen**

Capture audio and transcribe it to text (STT)

02

**Think**

Interpret intent, reason, and plan a response (LLM)

03

**Speak**

Generate a response and stream audio back (TTS)

## Two Main Architectures

**Cascaded Pipeline**

STT → LLM → TTS chained sequentially; modular and flexible, but adds latency at each step

**End-to-End (Native Voice)**

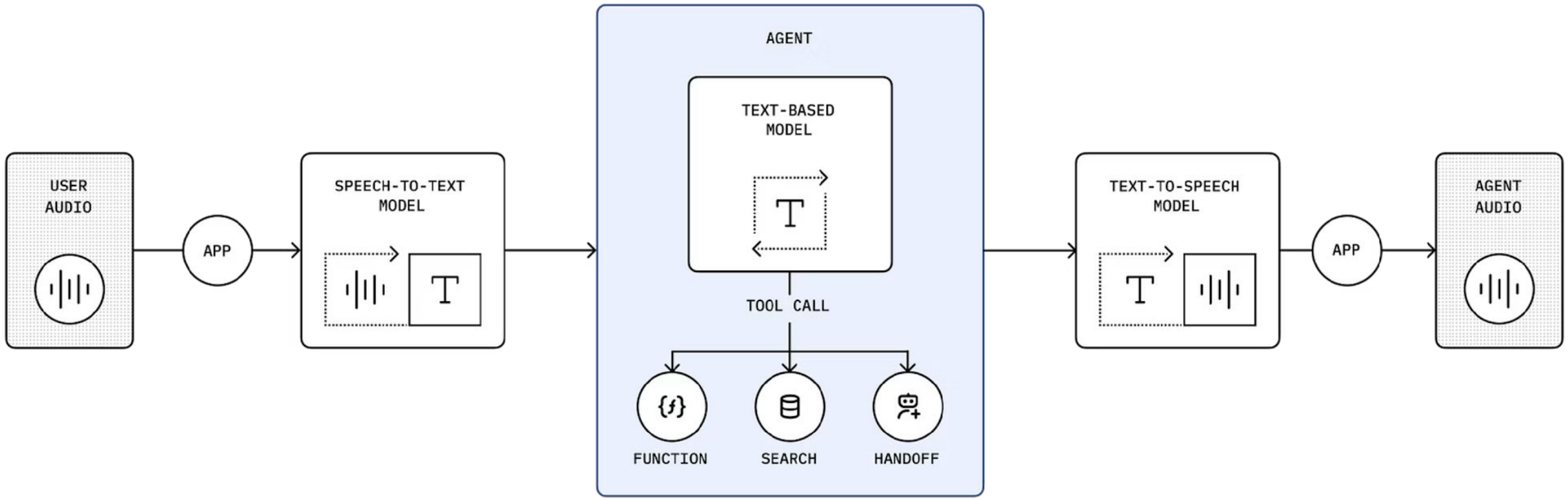
Audio in, audio out; the model handles everything natively with lower latency and more natural prosody

{ik} INTERVIEW KICKSTART

# Chained Architecture: The 'Sandwich'

Processes audio sequentially through three distinct stages.

**Examples:** Qwen3-TTS, Eleven Lab TTS & STT APIs, OpenAI Whisper, gpt TTS & STT models




{ik} INTERVIEW KICKSTART

# Chained Architecture: The 'Sandwich'

## Key Characteristics

| Strengths | Best For |
| --- | --- |
| High control and transparency | Structured workflows focused on specific user objectives |
| Robust function calling and structured interactions | Customer support |
| Reliable, predictable responses | Sales and inbound triage |
| Support for extended conversational context | Scenarios that involve transcripts and scripted responses |

## Analysis

### Pros

- Full control over each component — swap STT/TTS providers as needed
- Access to latest capabilities from modern text-modality models
- Transparent behavior with clear boundaries between components

### Cons

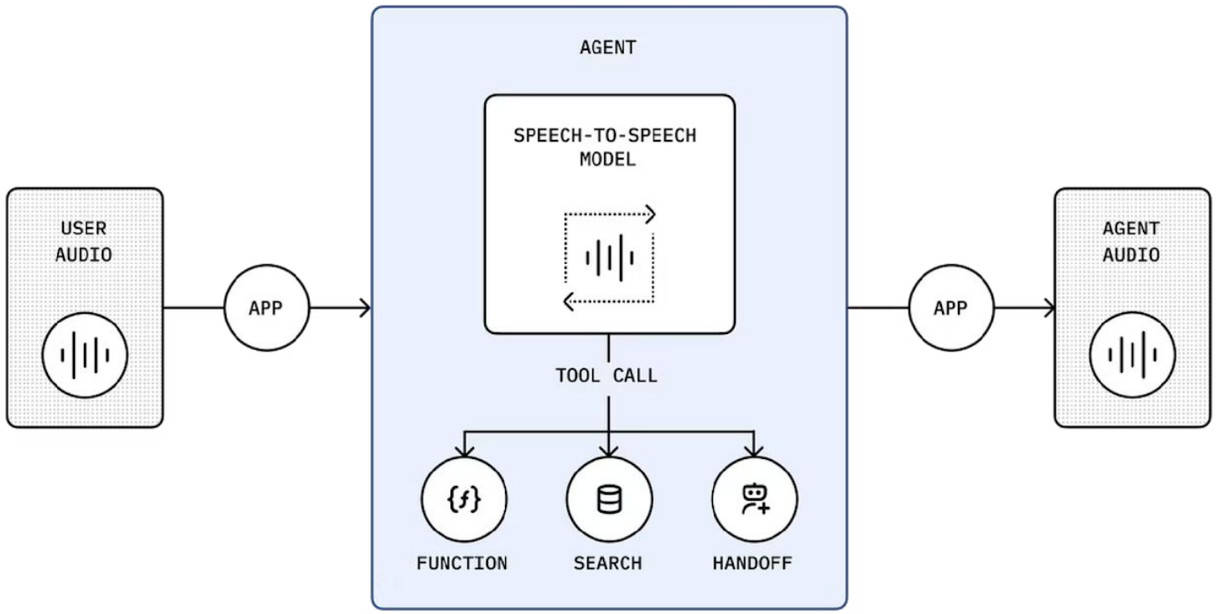
- Requires orchestrating multiple services
- Additional complexity in managing the pipeline
- Conversion from speech to text loses information (e.g., tone, emotion)

{ik} INTERVIEW KICKSTART

# Speech-to-Speech (Realtime) Architecture

A single multimodal model processes audio in and audio out — no transcription step, no pipeline.

**Example:** OpenAI gpt-realtime, Gemini 2.5 Flash, Qwen3-Omni, etc.




{ik} INTERVIEW KICKSTART

# Speech-to-Speech (Realtime) Architecture

## Key Characteristics

| Strengths | Best For |
| --- | --- |
| Low latency | Interactive, unstructured conversations |
| Rich multimodal understanding | Language tutoring & learning |
| Natural conversational flow | Conversational search & discovery |
| Vocal context understanding | Interactive customer service |

## Cons

- Limited model options, provider lock-in risk
- Less transparency in audio processing
- Reduced controllability & customization

{ik} INTERVIEW KICKSTART

# Subgraphs in LangGraph

A subgraph is a graph used as a node within another graph.

**Multi-Agent Systems**

Coordinate specialized agents as independent subgraphs within a larger orchestration

**Node Reuse**

Share and reuse a set of nodes across multiple graphs without duplication

**Distributed Development**

Teams work on separate subgraphs independently; as long as input/output schemas are respected, the parent graph needs no knowledge of internal details

{ik} INTERVIEW KICKSTART

# Subgraph Communication

When adding subgraphs, you need to define how the parent graph and the subgraph communicate.

| Pattern | When to Use | State Schemas |
| --- | --- | --- |
| Call a subgraph inside a node | Parent and subgraph have different state schemas (no shared keys), or you need to transform state between them | Write a wrapper function that maps parent state to subgraph input and subgraph output back to parent state |
| Add a subgraph as a node | Parent and subgraph share state keys — the subgraph reads from and writes to the same channels as the parent | Pass the compiled subgraph directly to add_node — no wrapper needed |

{ik} INTERVIEW KICKSTART

PART 1 — DEFINE THE SUBGRAPH

# Call a Subgraph Inside a Node

```
class SubgraphState(TypedDict):
    bar: str


def subgraph_node_1(state: SubgraphState):
    return {"bar": "hi! " + state["bar"]}


subgraph_builder = StateGraph(SubgraphState)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()
```

The subgraph has its own state schema (SubgraphState) — independent from the parent graph.

PART 2 — PARENT GRAPH WITH STATE TRANSFORMATION

# Call a Subgraph Inside a Node

```
class State(TypedDict):
    foo: str

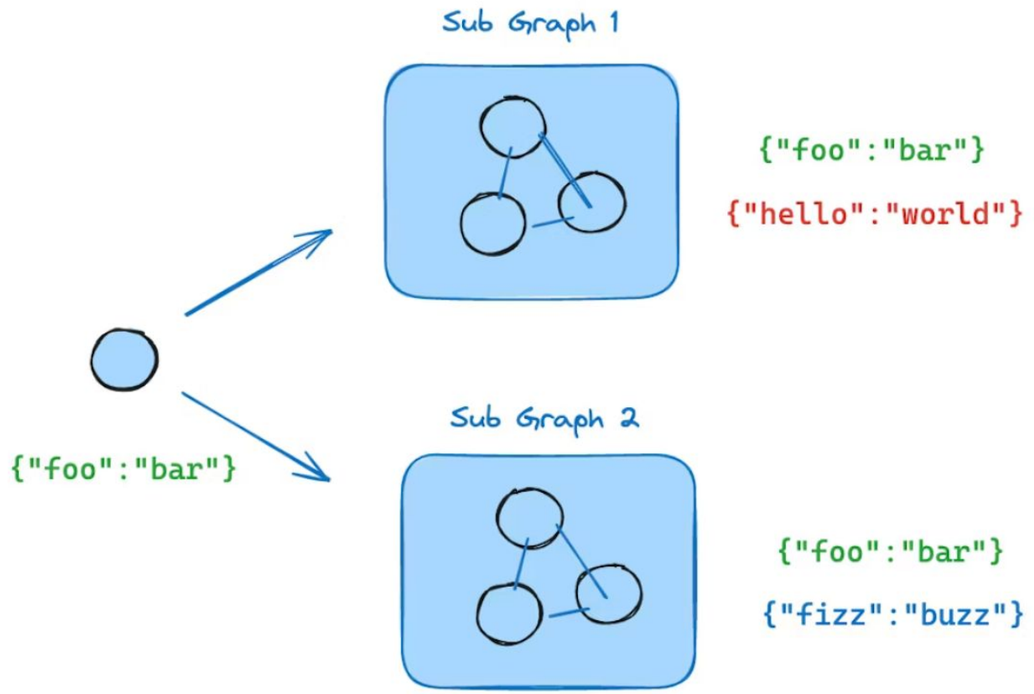
def call_subgraph(state: State):
    # Transform the state to the subgraph state
    subgraph_output = subgraph.invoke({"bar": state["foo"]})
    # Transform response back to the parent state
    return {"foo": subgraph_output["bar"]}

builder = StateGraph(State)
builder.add_node("node_1", call_subgraph)
builder.add_edge(START, "node_1")
graph = builder.compile()
```

The wrapper function maps parent state → subgraph input, then maps subgraph output → parent state.

# Add a Subgraph as a Node

When the parent graph and subgraph share state keys, pass the compiled subgraph directly to add_node — no wrapper function needed. The subgraph reads from and writes to the parent's state channels automatically.




{ik} INTERVIEW KICKSTART

# Add a Subgraph as a Node

SHARED STATE — DIRECT NODE ADDITION

```
class State(TypedDict):
    foo: str


# Subgraph
def subgraph_node_1(state: State):
    return {"foo": "hi! " + state["foo"]}


subgraph_builder = StateGraph(State)
subgraph_builder.add_node(subgraph_node_1)
subgraph_builder.add_edge(START, "subgraph_node_1")
subgraph = subgraph_builder.compile()
```

{ik} INTERVIEW KICKSTART

# Add a Subgraph as a Node

SHARED STATE — DIRECT NODE ADDITION

```
# Parent graph


builder = StateGraph(State)
builder.add_node("node_1", subgraph)
builder.add_edge(START, "node_1")
graph = builder.compile()
```

Both graphs share the same State schema — the subgraph is passed directly to add_node with no wrapper needed.

{ik} INTERVIEW KICKSTART

# Subgraph Persistence

By default, **subgraphs are stateless (no memory)**: each call starts with a blank slate. If a subagent needs multi-turn conversation memory (for example, a support assistant that builds context over several exchanges) **you can make it stateful (persistent memory)** so its conversation history and data accumulate across calls on the same thread.

**Stateless (Default)**

Each call starts fresh. Best for independent, single-turn tasks and most multi-agent systems.

**Stateful (Persistent Memory)**

Conversation history accumulates across calls on the same thread. Best for multi-turn interactions like support assistants.

**Example:** A customer support bot delegates to a specialist subagent — should it remember the customer's earlier questions, or start fresh each time it is called?

# Interrupts — Human In The Loop (HITL)

Interrupts allow you to pause graph execution at specific points and wait for external input before continuing. This enables human-in-the-loop patterns where you need external input to proceed. When an interrupt is triggered, LangGraph saves the graph state using its persistence layer and waits indefinitely until you resume execution.

**Pause & Review**

Stop the agent at any node to let a human inspect state before proceeding.

**Approve or Reject**

Human can approve the next action, modify state, or redirect the flow.

**Resume Execution**

Once approved, the graph continues from where it left off.

**Safety & Oversight**

Critical for high-stakes actions like sending emails, making purchases, or deploying code.

{ik} INTERVIEW KICKSTART

# Pause using interrupt

The interrupt function pauses graph execution and returns a value to the caller. When you call interrupt within a node, LangGraph saves the current graph state and waits for you to resume execution with input.

## To use interrupt, you need:

**A checkpointer to persist the graph state**
(use a durable checkpointer in production)

**A thread ID in your config**
so the runtime knows which state to resume from

**To call interrupt() where you want to pause**
(payload must be JSON-serializable)

{ik} INTERVIEW KICKSTART

# When you call interrupt, here's what happens:

**Graph execution gets suspended**

at the exact point where interrupt is called

**State is saved**

using the checkpointer so execution can be resumed later. In production, this should be a persistent checkpointer (e.g. backed by a database)

**Value is returned**

to the caller under `__interrupt__`; it can be any JSON-serializable value (string, object, array, etc.)

**Graph waits indefinitely**

until you resume execution with a response

**Response is passed back**

into the node when you resume, becoming the return value of the interrupt() call

{ik} INTERVIEW KICKSTART

# Resuming Interrupts

**Same Thread ID**

You must use the same thread ID when resuming that was used when the interrupt occurred.

**Resume Value**

The value passed to Command(resume=...) becomes the return value of the interrupt call.

**Node Restarts**

The node restarts from the beginning of the node where the interrupt was called when resumed, so any code before the interrupt runs again.

**JSON-Serializable**

You can pass any JSON-serializable value as the resume value.

{ik} INTERVIEW KICKSTART

# Common Patterns

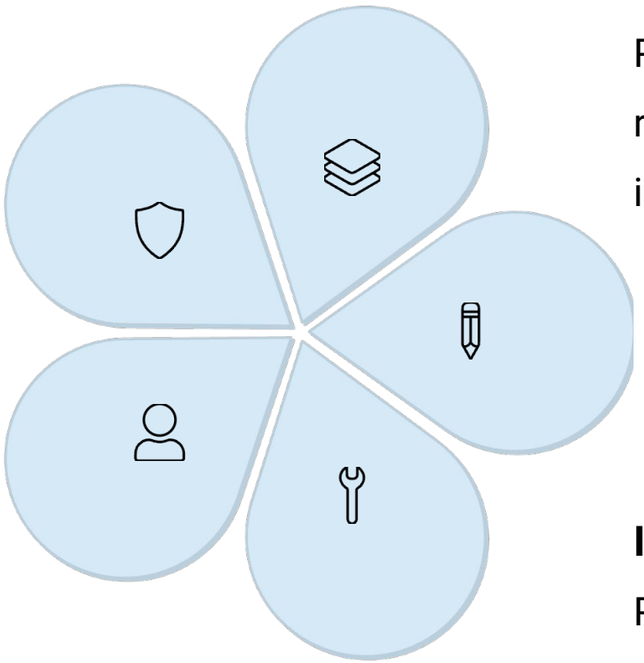
The key thing that interrupts unlock is the ability to pause execution and wait for external input. This is useful for a variety of use cases, including:

**Approval Workflows**

Pause before executing critical actions (API calls, database changes, financial transactions)

**Validating Human Input**

Pause before proceeding to the next step to validate human input



**Handling Multiple Interrupts**

Pair interrupt IDs with resume values when resuming multiple interrupts in a single invocation

**Review and Edit**

Let humans review and modify LLM outputs or tool calls before continuing

**Interrupting Tool Calls**

Pause before executing tool calls to review and edit the tool call before execution

{ik} INTERVIEW KICKSTART

# Break Time!

Take a 10-minute pause to recharge.



### Enjoy a Warm Drink

Sip on your favorite beverage.



### Set Your Timer

You'll be refreshed in 10 minutes.



### Find Your Zen

Breathe and clear your mind.

{ik} INTERVIEW KICKSTART

# Let's Get Your Hands Dirty with Code

Time to build! We'll walk through creating a multi-agent system step by step.



{ik} INTERVIEW KICKSTART

# What we're building?

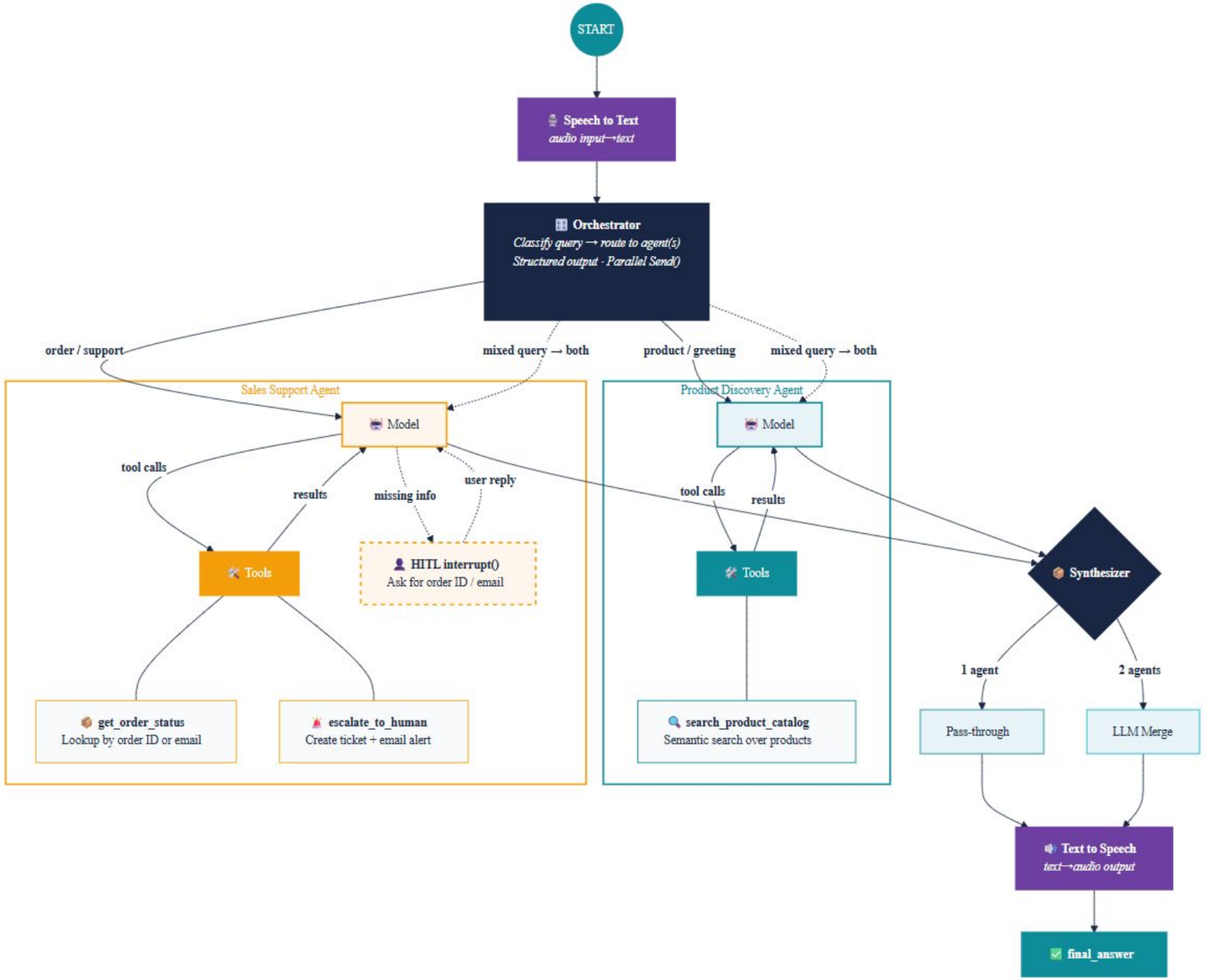
An E-Commerce AI Assistant

A **multi-modal conversational agentic system** for e-commerce customer support, built with LangGraph, OpenAI Whisper (STT), and OpenAI TTS.




{ik} INTERVIEW KICKSTART

# What We're Building




{ik} INTERVIEW KICKSTART

# Project Setup

1. **Install UV — An extremely fast Python package and project manager**

   Refer to: https://docs.astral.sh/uv/getting-started/installation/

2. **Create a Project — Set up your UV project with Python version management**

   Refer to: https://docs.astral.sh/uv/guides/projects/#python-version

{ik} INTERVIEW KICKSTART

# Instruction Guide

Refer to: Multimodal_Conversational_System_Instructor_Guide

{ik} INTERVIEW KICKSTART



♡ APPRECIATION ✨ GRATITUDE

# Thank You!

Questions? Let's Discuss!

{ik} INTERVIEW KICKSTART